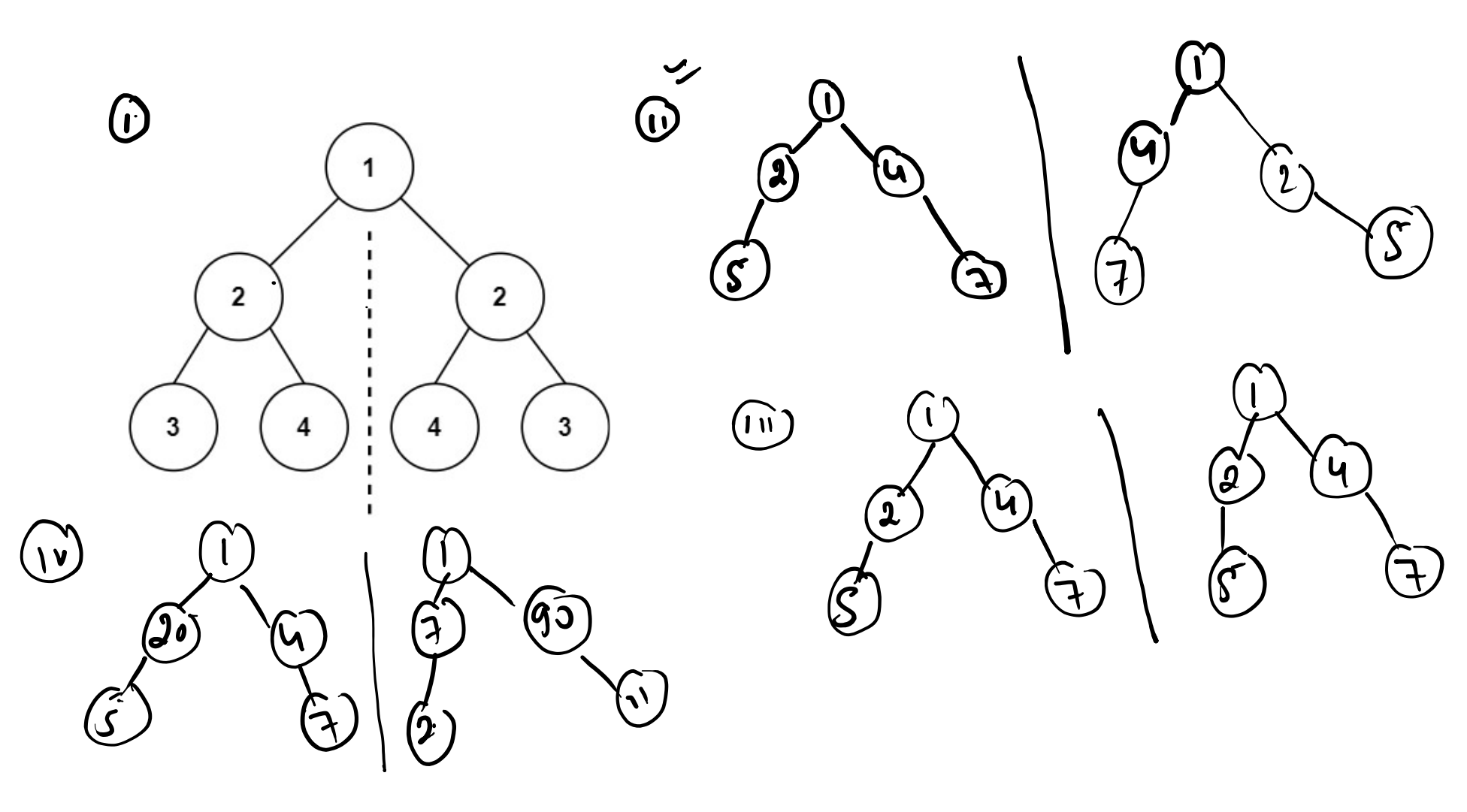
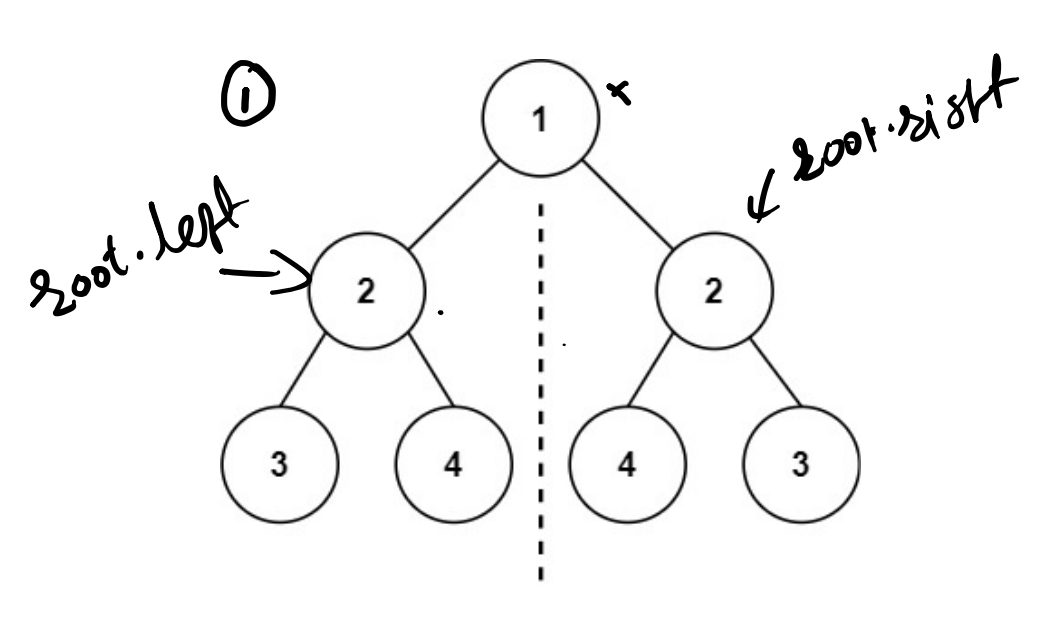
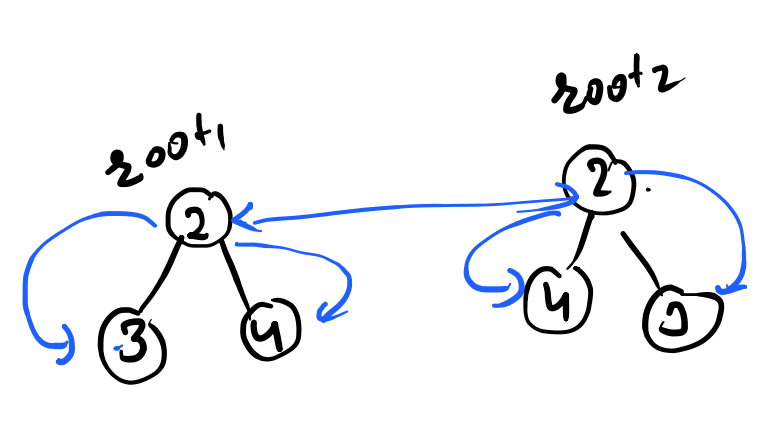
Lec-21

14/07/26 10:21 AM



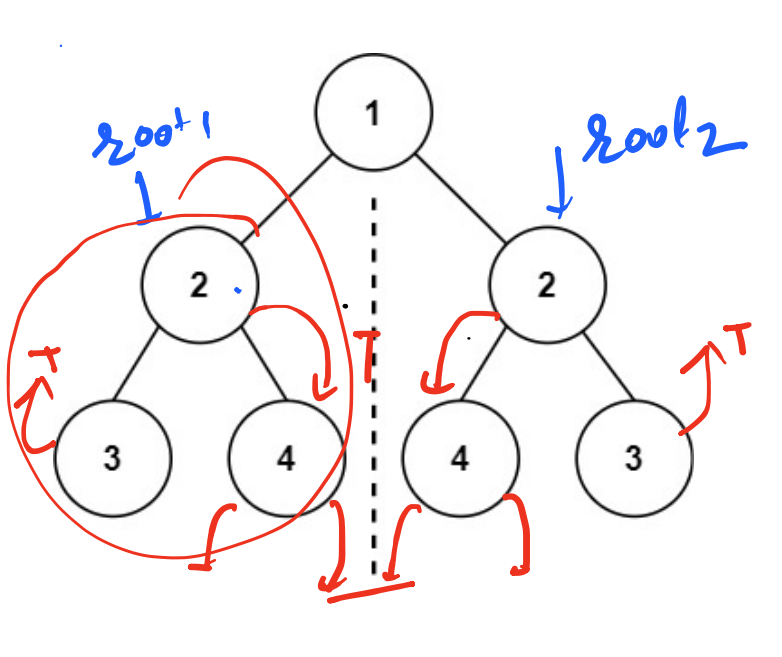

root.left
root.right

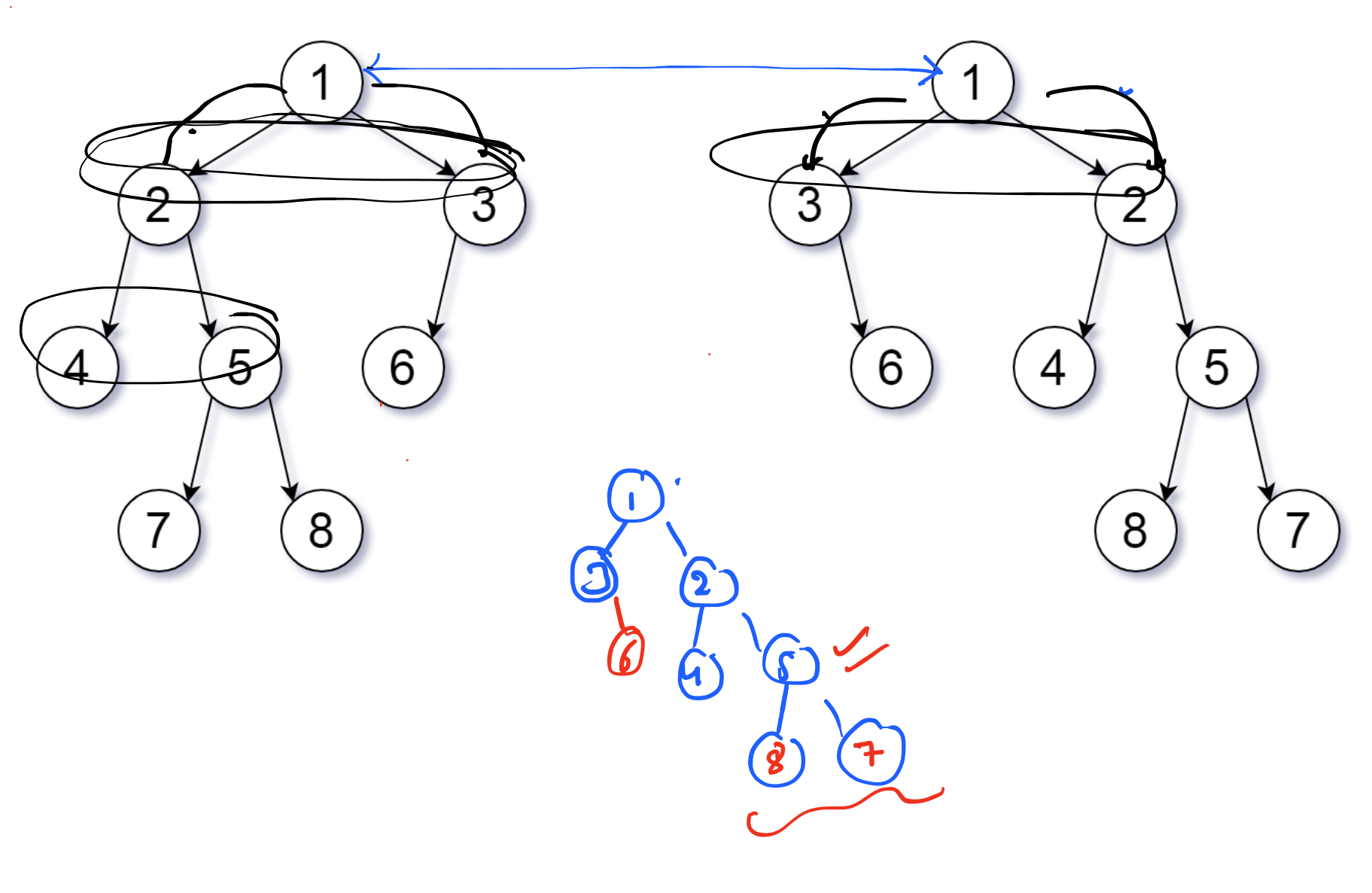

root1
root2

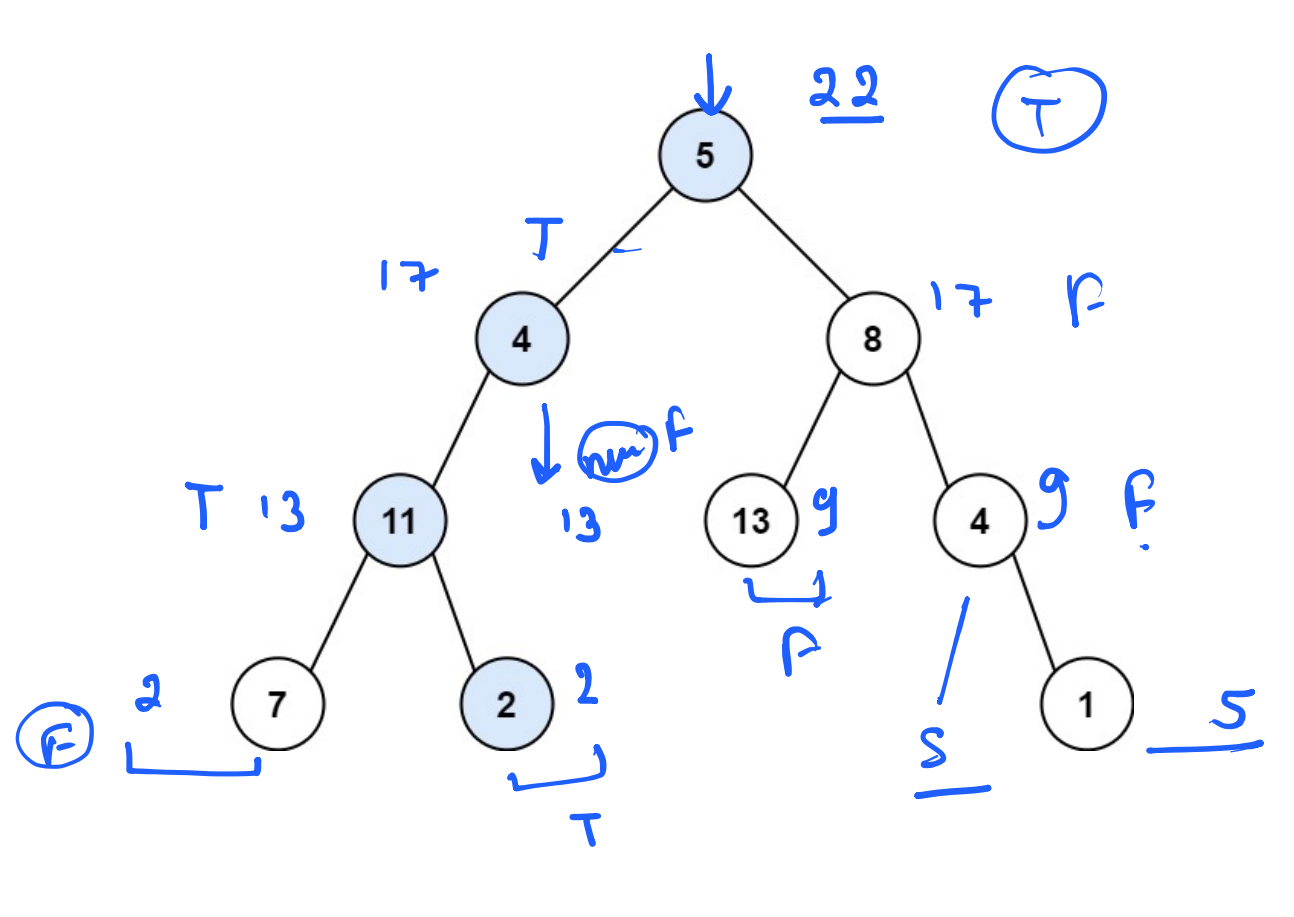
root1==null && root2==null
=
root1 ==null || root2==null x
root1.val != root2.val

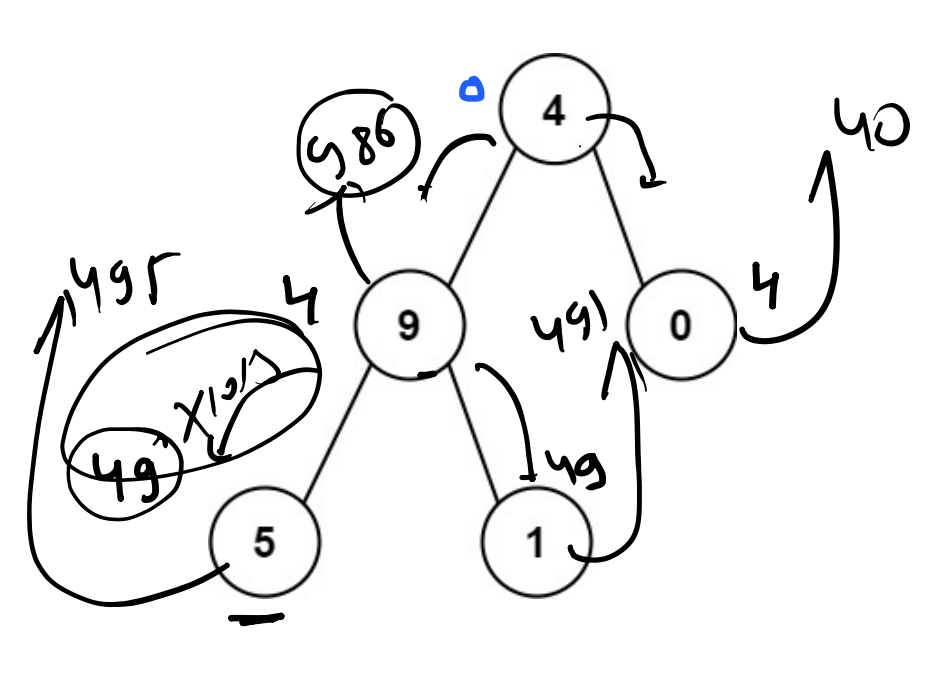
```
public boolean Symmetric(TreeNode root1, TreeNode root2) {
        if (root1.left == null && root2.right == null) {
            return true;
        }
        if (root1.left == null || root2.right == null) {
            return false;
        }
        if (root1.val != root2.val) {
            return false;
        }
        boolean left = Symmetric(root1.left, root2.right);
        boolean right = Symmetric(root1.right, root2.left);
        return left && right;
    }
```

Left
Right


root1
root2
T
T



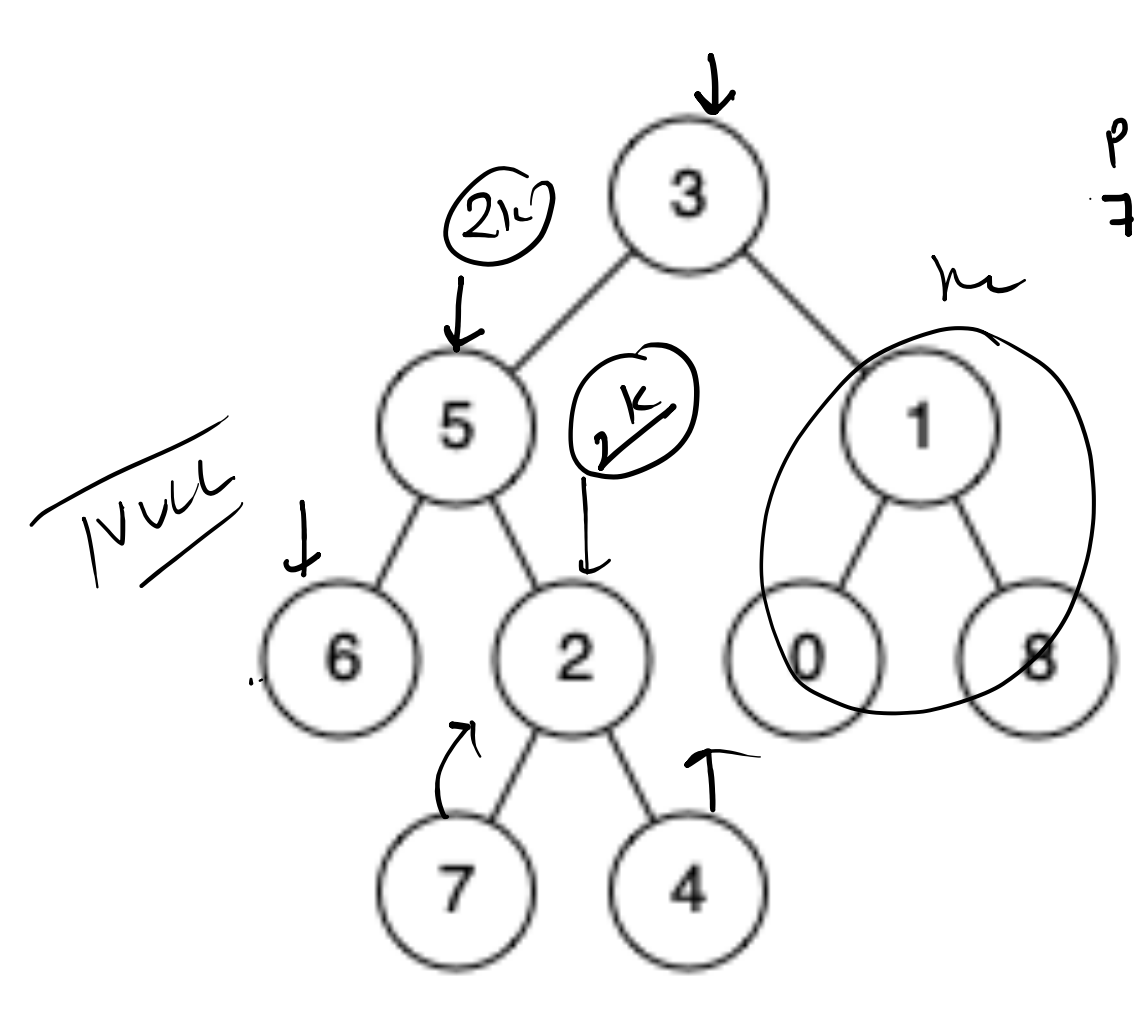

22
T
17
T
17
F
T 13
13
F
9
F
2
2
F
T
S
5


986
0
40
495
4
491
4
49
491
49

495 + 491 + 40

(495, 491, 40)

1026



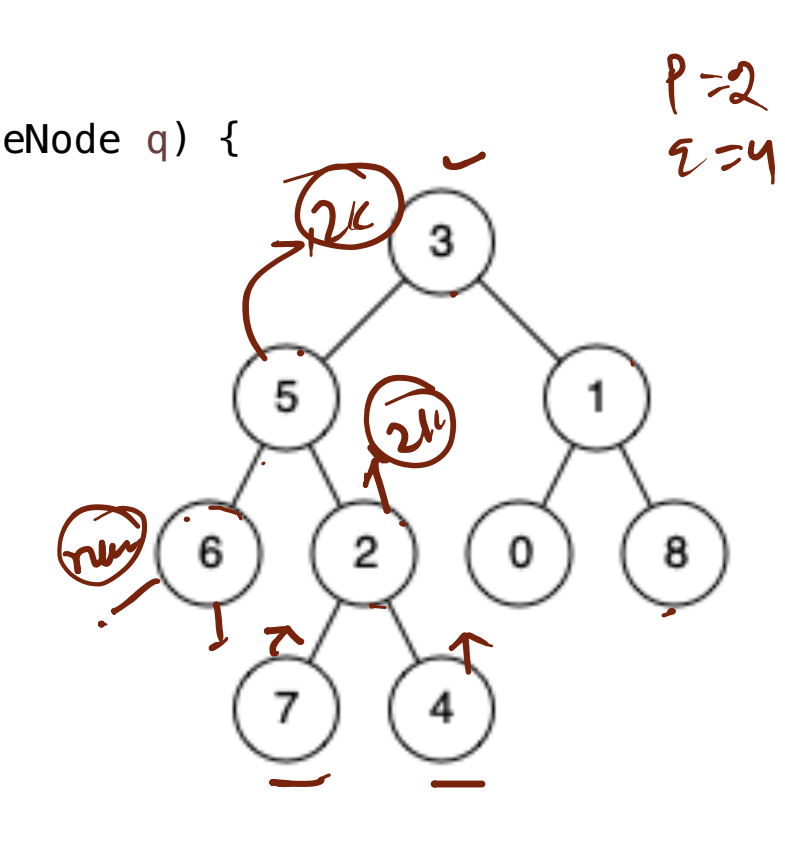
p q
7, 4

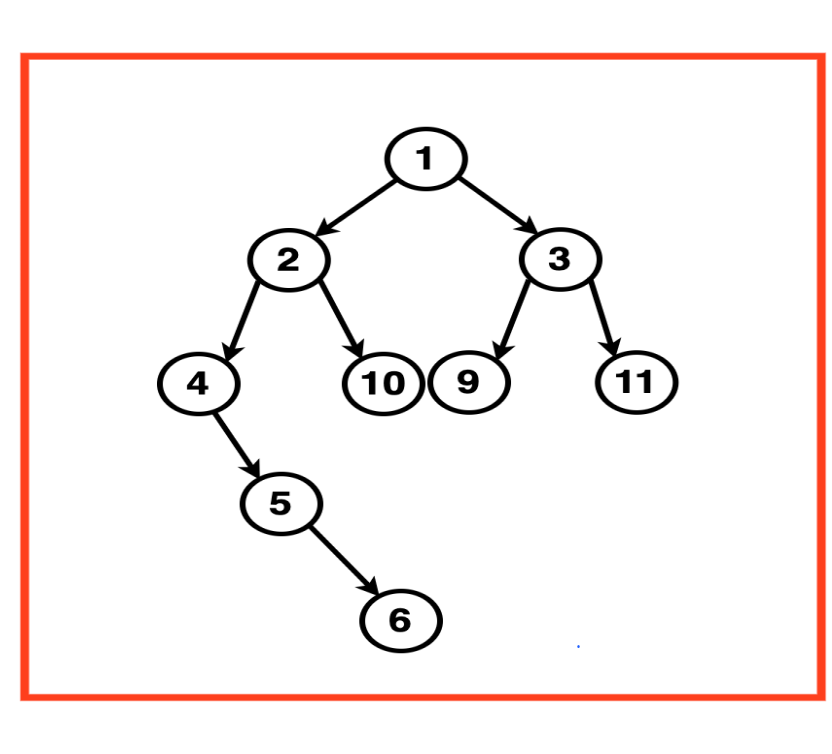
3 5 2 7
3 5 2 4

3 5 2
3 5 2 4

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
            if (root == null) {
                return null;
            }
            if (root == p || root == q) {
                return root;
            }

            TreeNode left = lowestCommonAncestor(root.left, p, q);
            TreeNode right = lowestCommonAncestor(root.right, p, q);
            if(left!=null && right!=null) {
                return root;
            }
            else if(left==null) {
                return right;
            }
            else {
                return left;
            }

        }
```

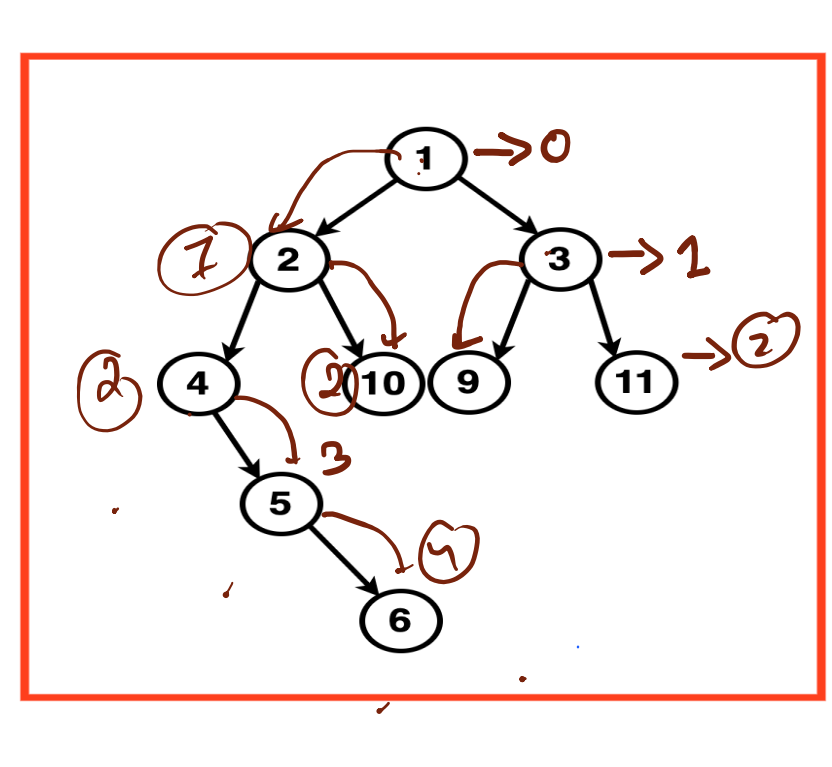
p=2
q=4





[1,3,11,9,6]

1
2 3
4 10 9 11
5
6

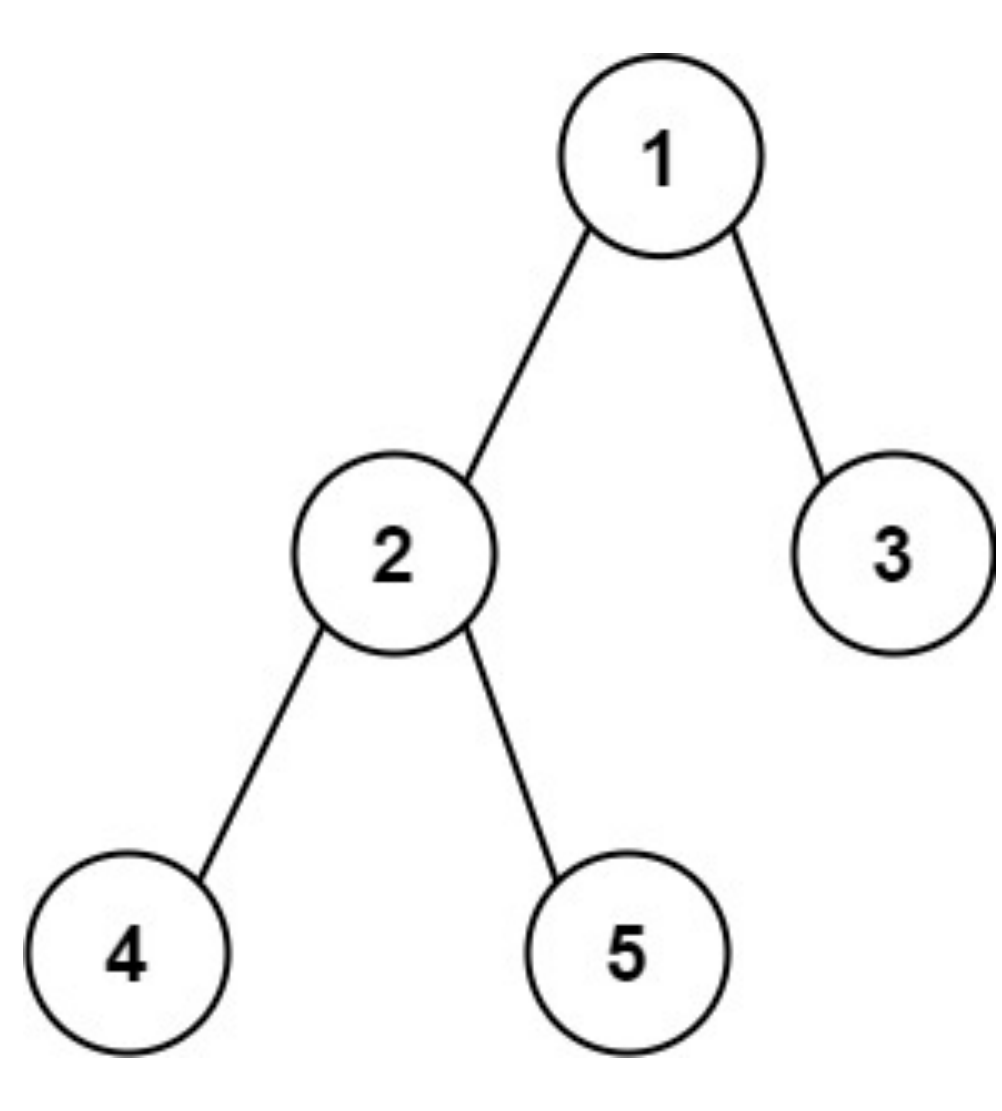


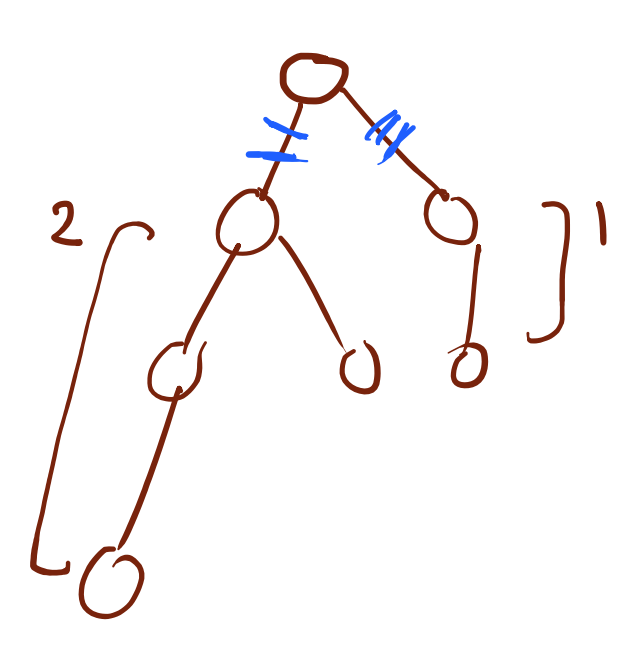
[1, 3, 11, 5, 6

[1,3,11,5,6)

maxdepth = -1
1 0
2 3 4

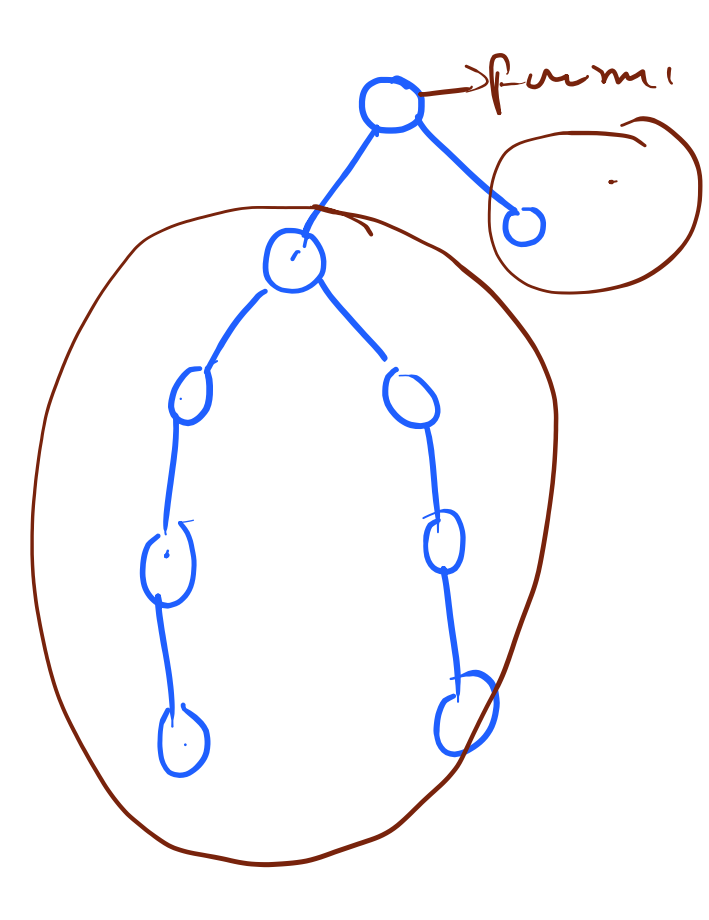
currlevel > maxdepth





lh+rh+2

ld = lh+rh+2



3+0+2=5

2+2+2=6

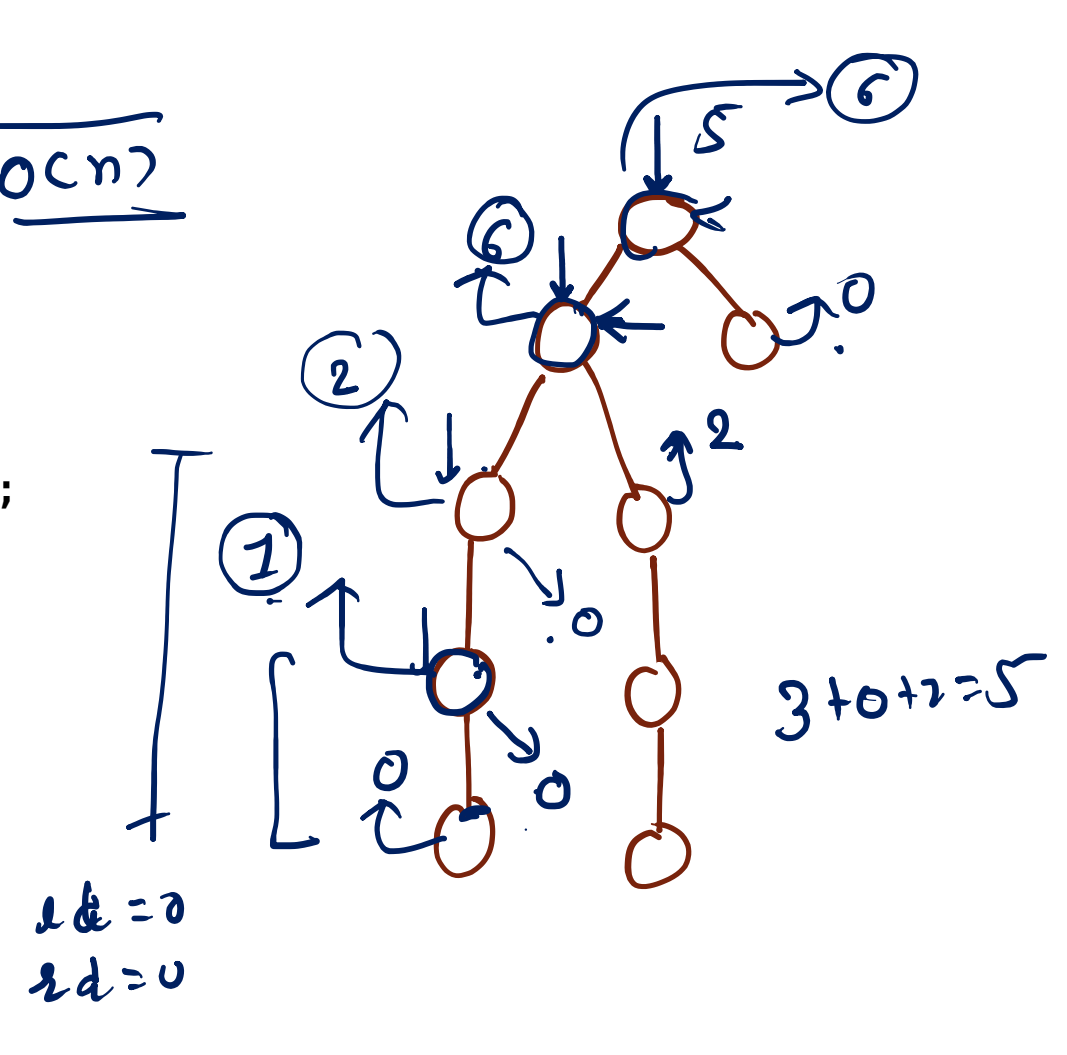
```
public int diameterOfBinaryTree(TreeNode root) {
            if(root==null) {
                return 0;
            }

            int ld = diameterOfBinaryTree(root.left);
            int rd = diameterOfBinaryTree(root.right);
            int sd=ht(root.left)+ht(root.right)+2;
            return Math.max(sd, Math.max(rd, ld));
        }
```

n²

O(n)

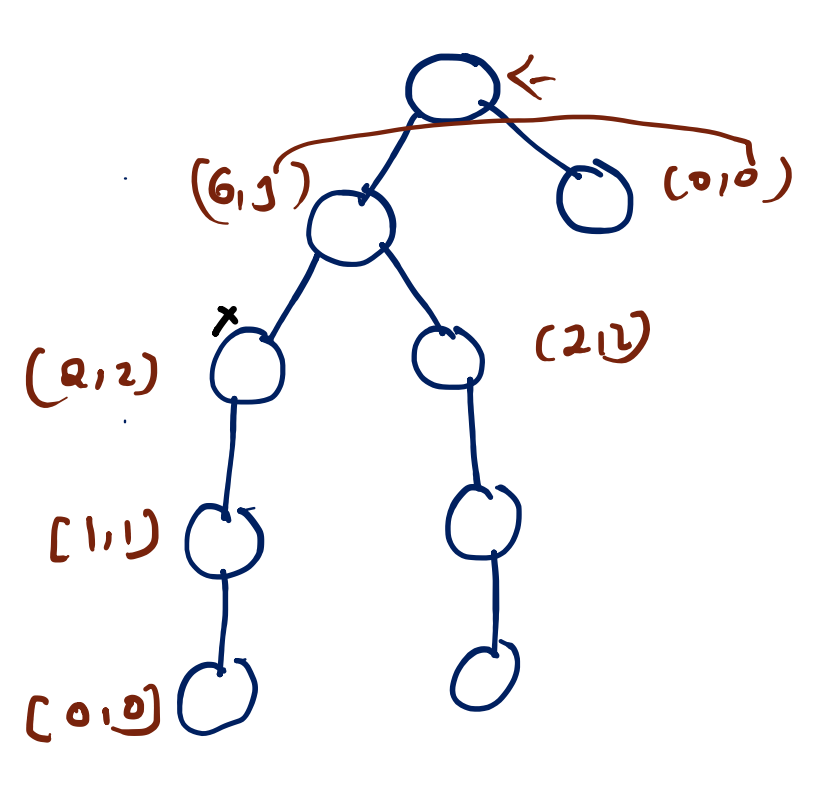
-1 -1 + 2 = 0
0 + -1 + 2 = 1

1-1+2 = 2
2+2+2=6


5
6
6
0
2
2
1
0
0
0

ld=0
rd=0

3+0+2=5


(6,3)
(0,0)
(2,2)
(2,2)
(1,1)
(0,0)

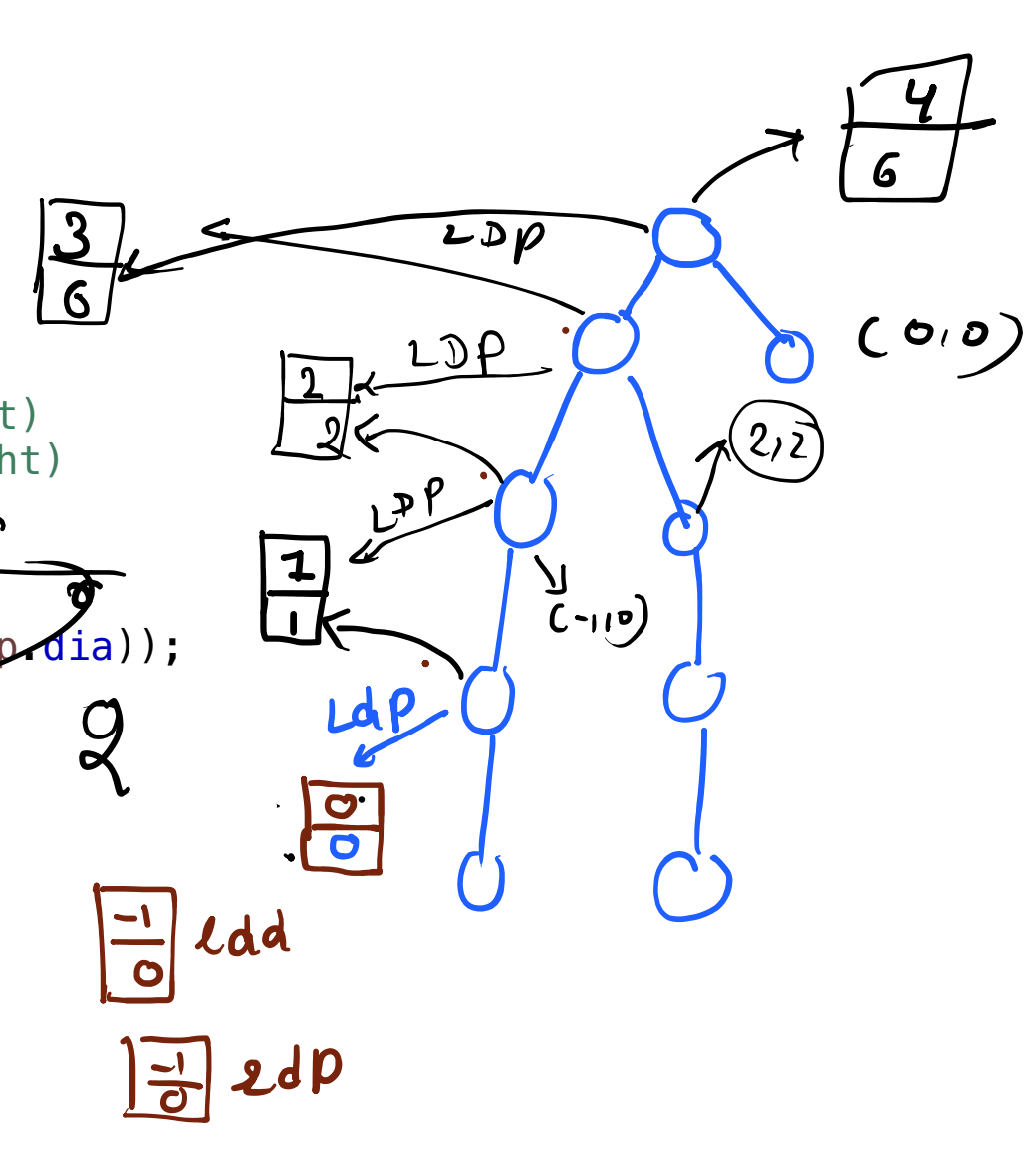
```
public DiaPair diameter(TreeNode root) {
        if (root == null) {
            return new DiaPair();
        }
        DiaPair ldp = diameter(root.left);// (dia,ht)
        DiaPair rdp = diameter(root.right);// (dia,ht)
        DiaPair sdp = new DiaPair();
        int sd = ldp.ht + rdp.ht + 2;
        sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
        sdp.dia = Math.max(sd, Math.max(ldp.dia, rdp.dia));
        return sdp;
    }
```

3,+0+2=5

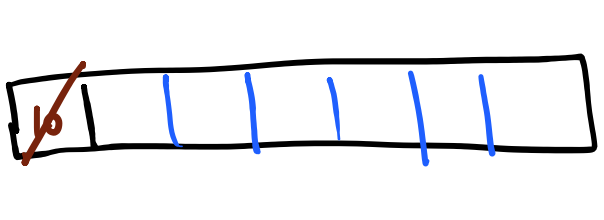
5 0 2 9

0 -1 +2 = 1

-1 -1 +2=0

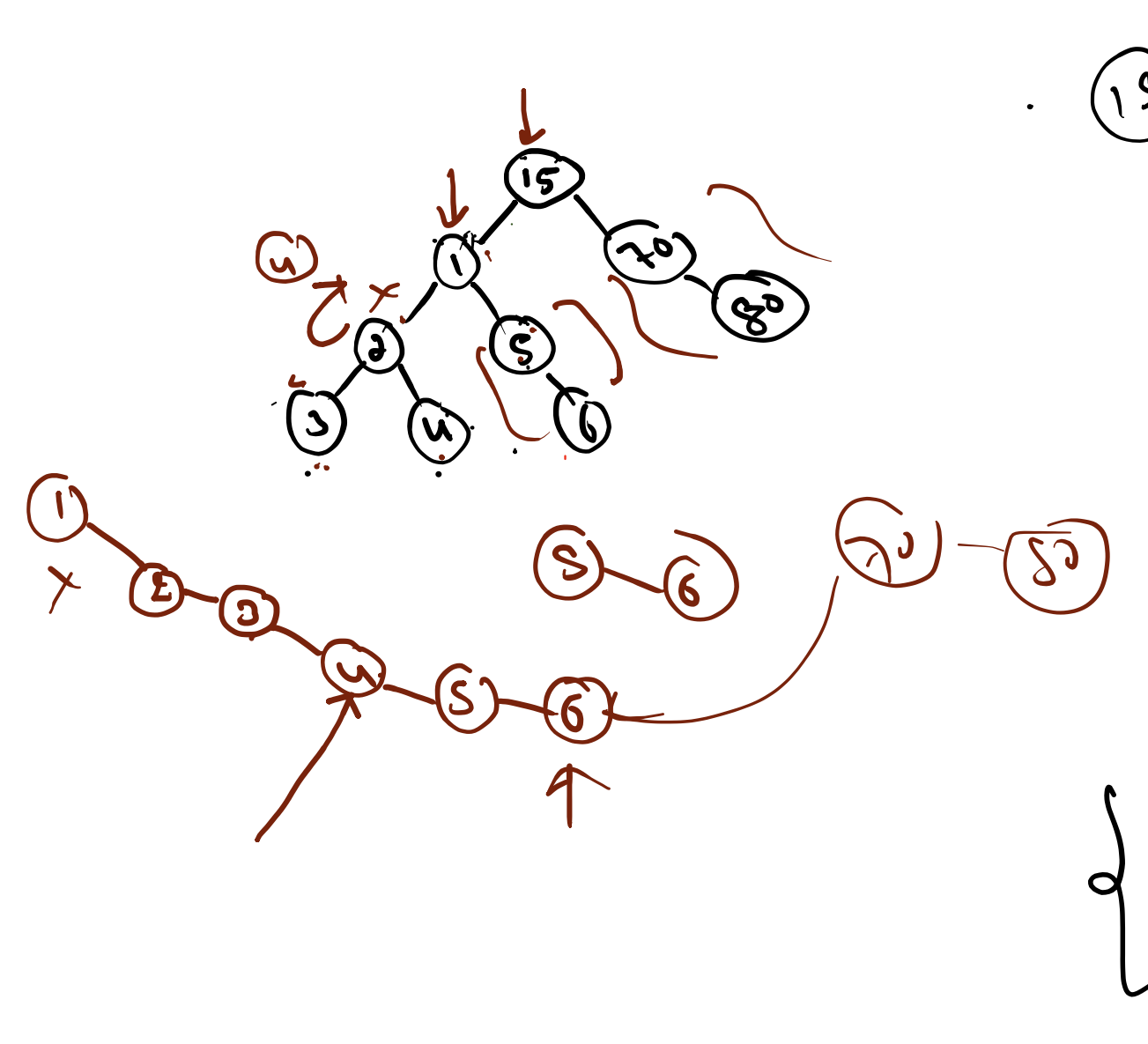

4
6
3
6
LDP
(0,0)
2
2
LDP
2,2
LDP
1
1
(-1,1,0)
LDP
0
0
-1
0
ldd
-1
0
rdp

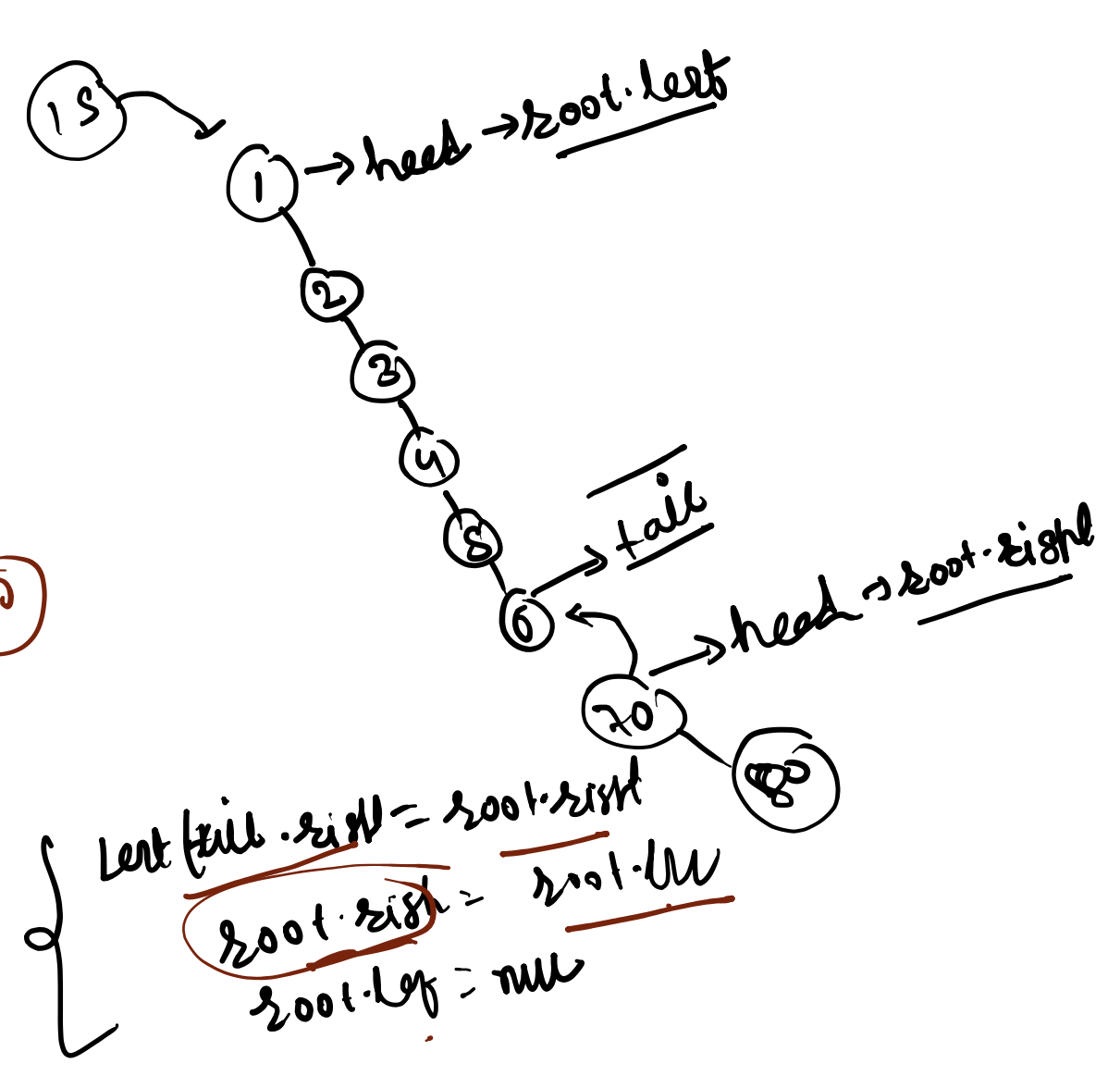
(10, 5, 7, 13, 8, -1, 7, -1,-1,-1,-1 31,-1,-1-1]



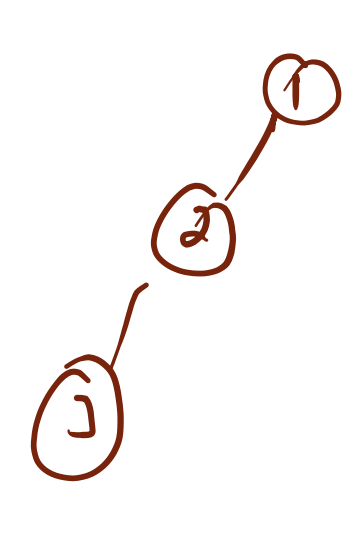
```
private void createTree() {
        // TODO Auto-generated method stub
        Scanner sc = new Scanner(System.in);
        Queue<Node> q = new LinkedList<>();
        int item =sc.nextInt();
        Node nn = new Node(item);
        root=nn;
        q.add(nn);
        while(!q.isEmpty()) {

        }

    }
```

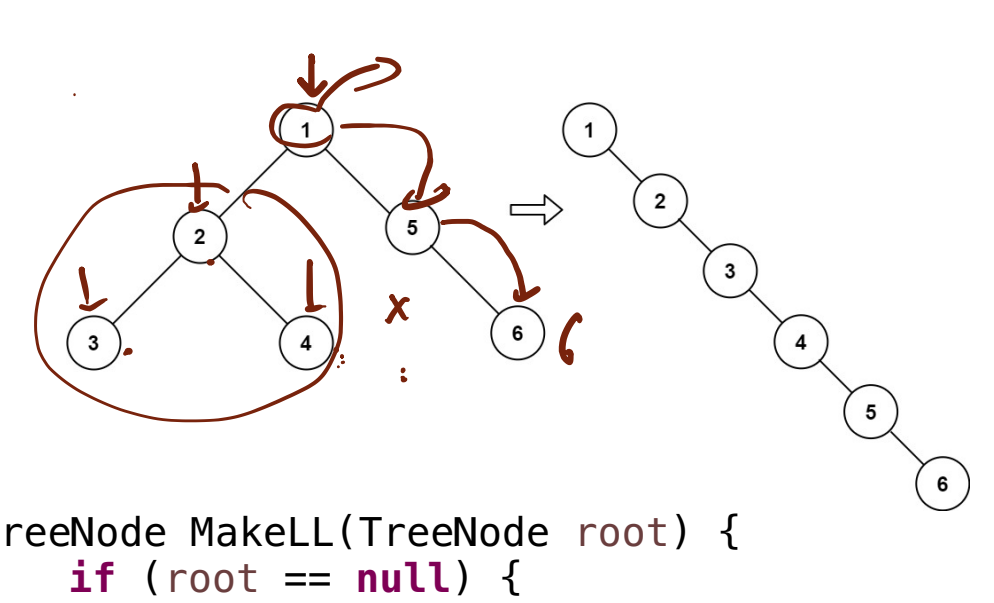
10

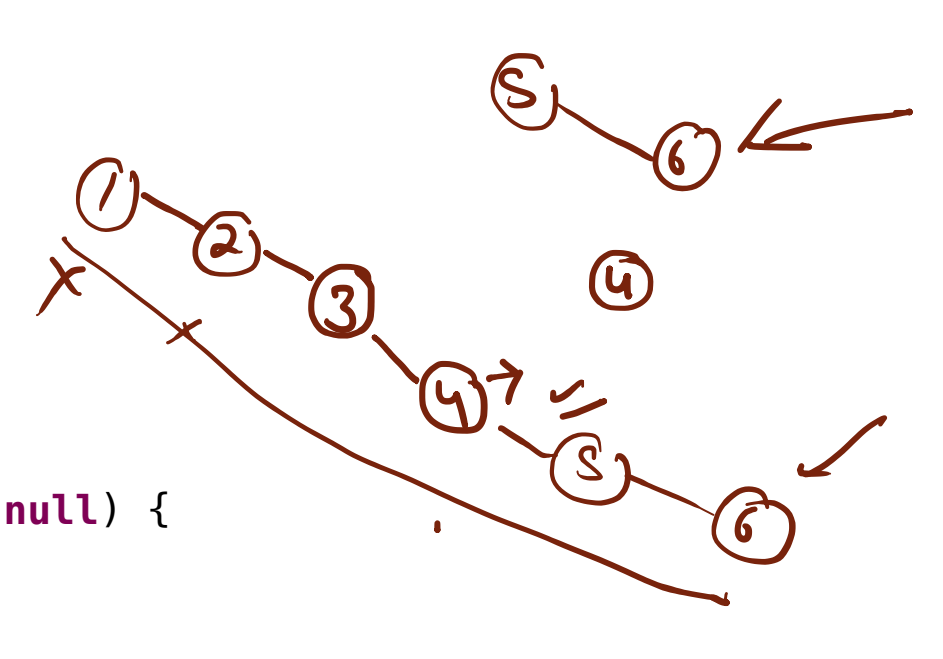

15
20
80
1
4
2
5
3
4
6
1
2
3
4
5
6
5
6
20
80


15
1
head → root.left
2
3
4
5
tail
6
head → root.right
20
80

left_tail.right = root.right
root.right = root.left
root.left = null

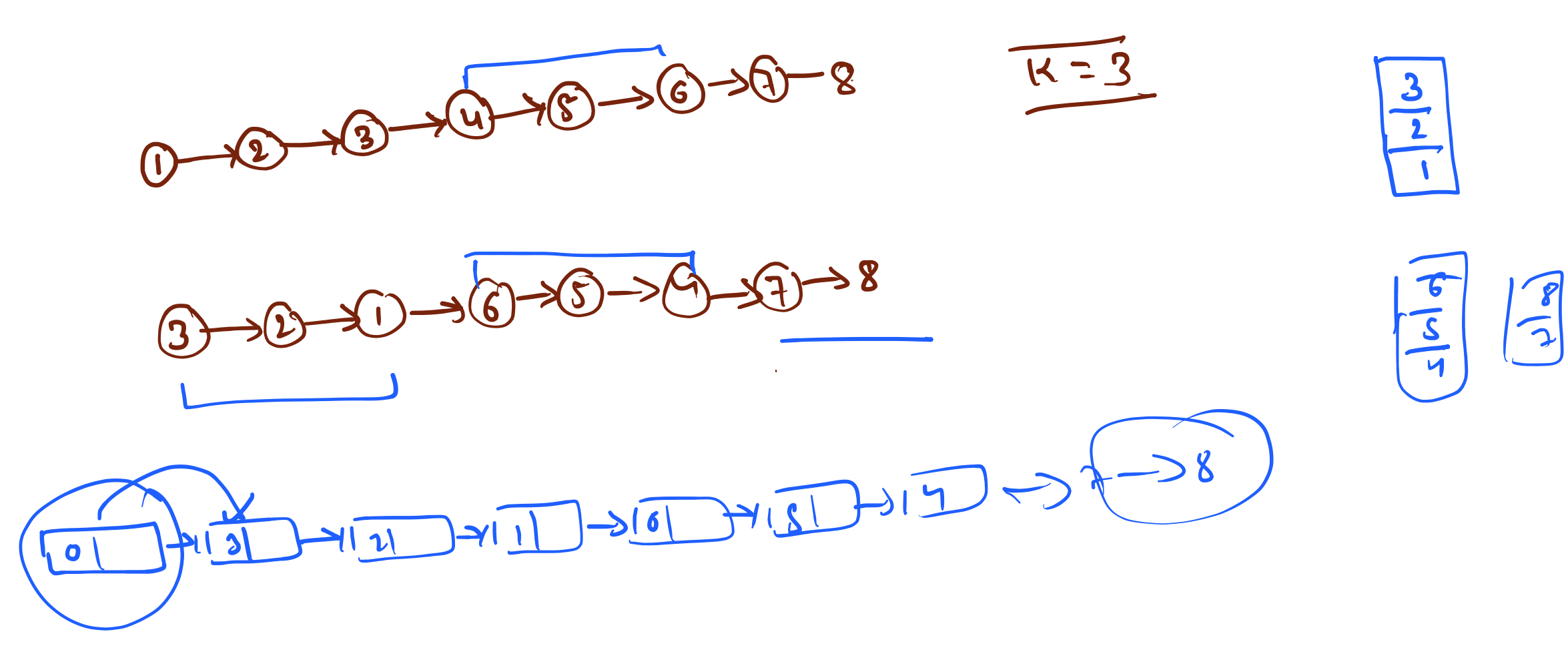






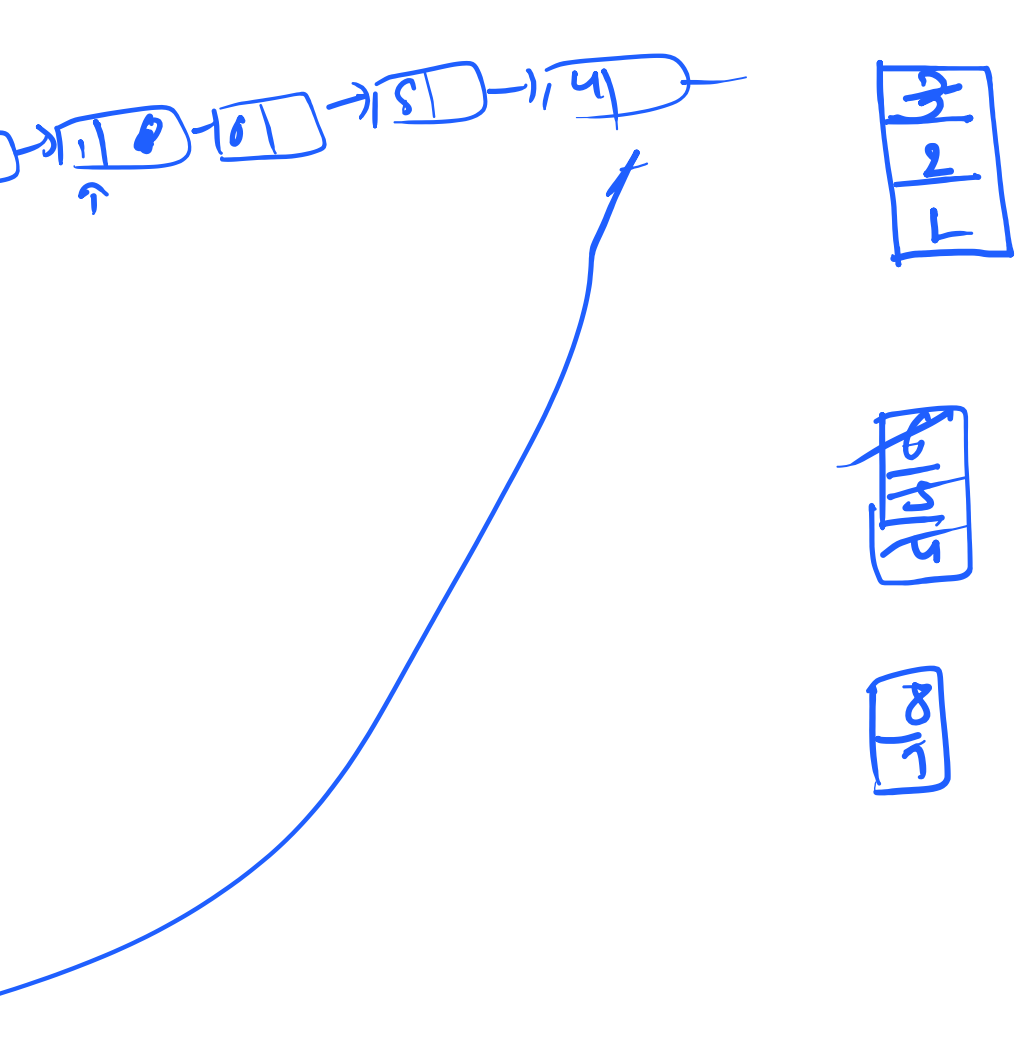
```
public TreeNode MakeLL(TreeNode root) {
        if (root == null) {
            return null;
        }
        if (root.left == null && root.right == null) {
            return root;
        }

        TreeNode left_Tail = MakeLL(root.left);
        TreeNode right_Tail = MakeLL(root.right);
        if (left_Tail != null) {
            left_Tail.right = root.right;
            root.right = root.left;
            root.left = null;
        }
        return right_Tail != null ? right_Tail : left_Tail;
    }
```

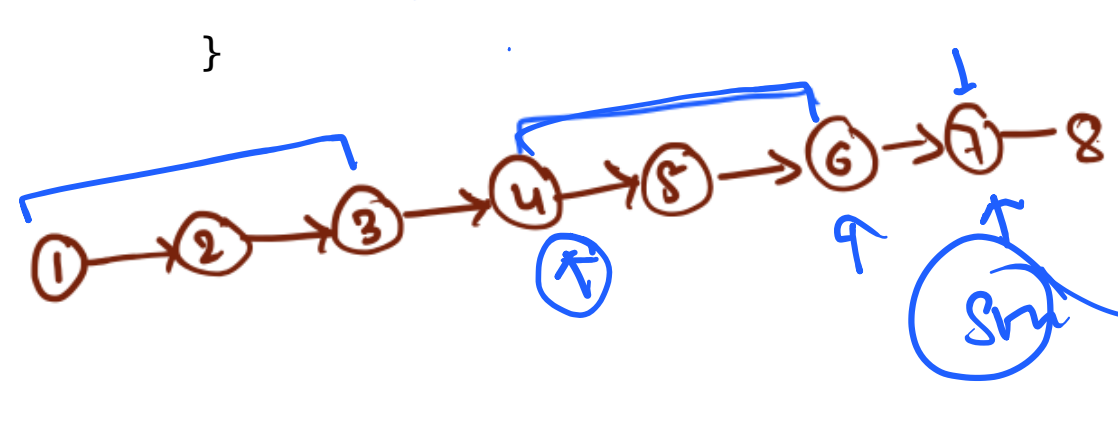
K = 3


1
2
3
4
5
6
7
8
3
2
1
3
2
1
6
5
4
7
8
6
5
4
8
7
0
3
2
1
6
5
4
7
8

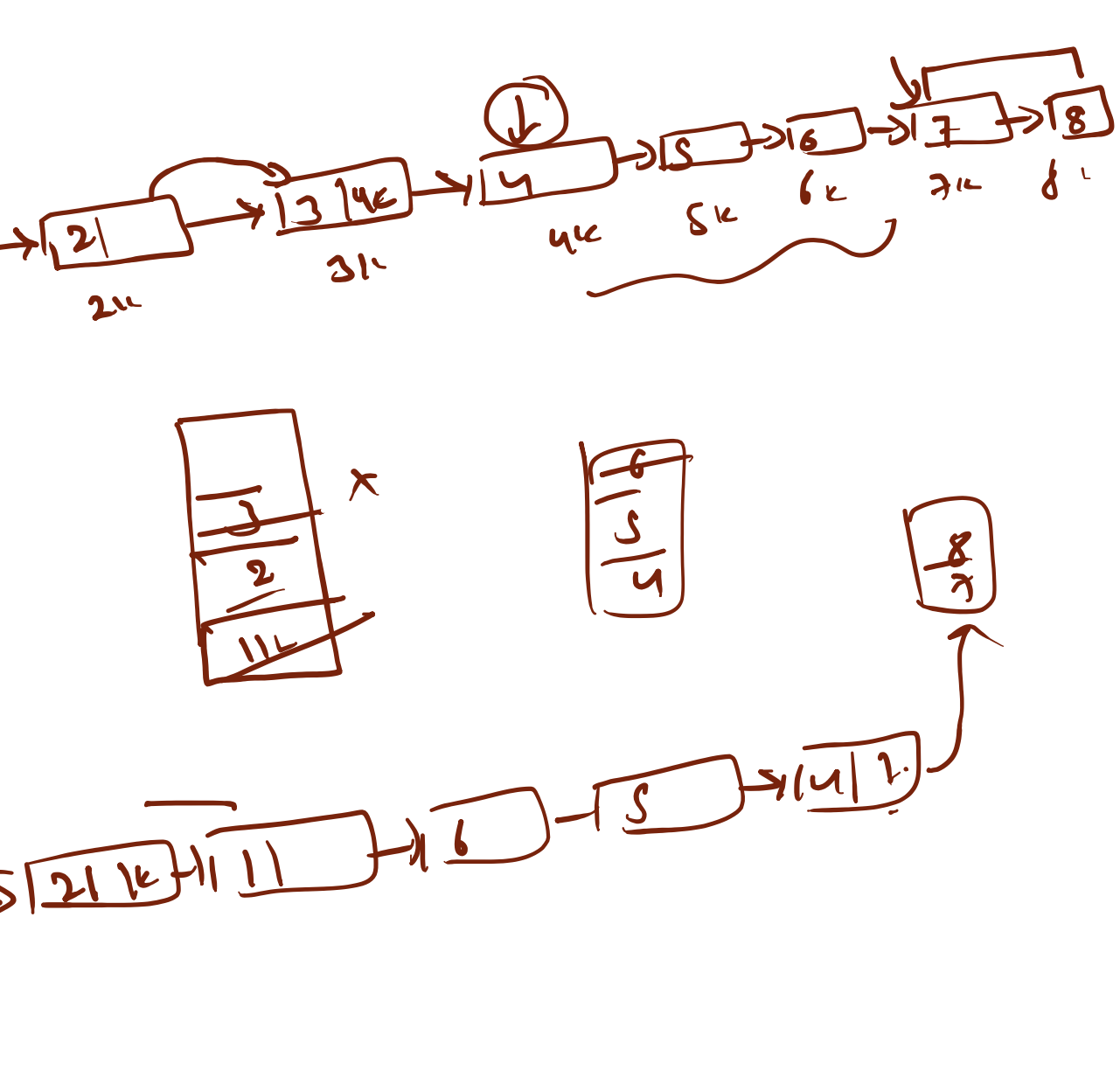
```
public ListNode reverseKGroup(ListNode head, int k) {
        Stack<ListNode> st = new Stack<>();
        ListNode Dummy = new ListNode();
        ListNode temp = Dummy;
        while (head != null) {
            st.push(head);
            if (st.size() == k) {
                while (!st.isEmpty()) {
                    Dummy.next = st.pop();
                    Dummy = Dummy.next;
                }
            }
            head = head.next;
        }

    }
```

Start=head


0
3
2
1
6
5
4
3
2
1
6
5
4
8
7


1
2
3
4
5
6
7
8
Start

```
public ListNode reverseKGroup(ListNode head, int k) {
        Stack<ListNode> st = new Stack<>();
        ListNode Dummy = new ListNode();
        ListNode start = head;
        ListNode temp = Dummy;
        while (head != null) {
            st.push(head);
            head = head.next;
            if (st.size() == k) {
                while (!st.isEmpty()) {
                    Dummy.next = st.pop();
                    Dummy = Dummy.next;
                }
                start = head;
                Dummy.next = null;
            }
        }
        if (!st.isEmpty()) {
            Dummy.next = start;
        }
        return temp.next;
    }
```


1
1k
2
2k
3
4k
3k
4
4k
5
5k
6
6k
7
7k
8
8
3
2
1k
6
5
4
8
7
3k
3
2k
2
1k
1
6
5
4
7